

DOCUMENTATION GESTION DE PROJET

V i t e & G o u r m a n d

Projet ECF — Application web et web mobile de traiteur événementiel

Titre professionnel Graduate Développeur Web et Web Mobile

Sofiène Guernane
Septembre 2026

Sommaire

1. Contexte et Organisation du Projet
2. Méthodologie et Suivi (Trello)
3. Stratégie de Versioning et Gestion Git
4. Gestion des risques techniques en déploiement
5. Rétroplanning et Phases de Développement
6. Gestion des Risques et Solutions Apportées

1. Contexte et Organisation du Projet

Ce projet a été conduit en parallèle d'une activité professionnelle de chef d'entreprise. Cette double charge a nécessité une rigueur absolue dans la gestion du temps, l'organisation des tâches et un investissement personnel soutenu, notamment en soirées et le weekend.

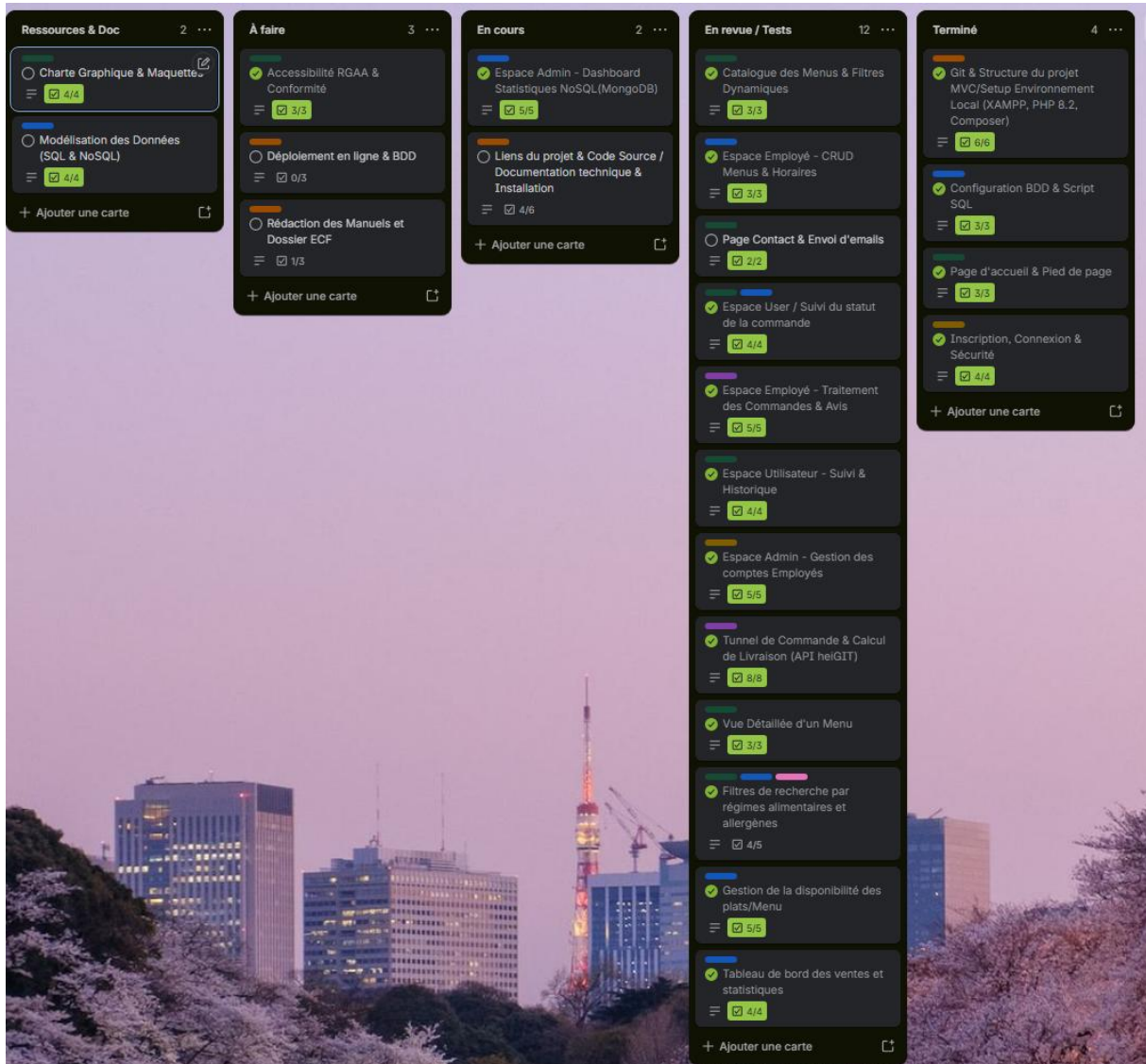
Au-delà de cette contrainte d'agenda, le projet a connu une inflexion majeure en cours de formation. Les débuts avaient déjà été ralentis par la double casquette entrepreneur/apprenant, le développement étant parti de zéro sans base préexistante à réutiliser. Après plusieurs reports de deadline, alors que le développement d'un premier sujet était avancé à environ 70 %, l'organisme de formation a modifié les consignes de l'ECF, nécessitant une refactorisation complète et un pivot vers le sujet Vite & Gourmand.

Cette situation a exigé une réorganisation immédiate du planning. Afin d'honorer la date de rendu sans compromettre la qualité du livrable, les briques logiques génériques déjà développées (le routeur, la structure MVC et les couches de sécurité) ont été réutilisées de manière pragmatique, tandis que le reste de l'application a été reconstruit pour répondre précisément aux nouvelles exigences du cahier des charges.

Cette réorganisation a notamment conduit à mettre temporairement en pause le suivi Trello durant la phase critique de refactorisation, la priorité ayant été donnée à la livraison du code dans les délais impartis. Le tableau a été reconstitué a posteriori pour refléter fidèlement les tâches réellement effectuées durant cette période.

2. Méthodologie et Suivi (Trello)

Pour structurer ce développement dans un délai contraint, une démarche agile de type Kanban a été adoptée, l'outil Trello a servi de tableau de bord central pour le suivi des tâches :



(<https://trello.com/b/qGLPe8mZ>)

- **Ressources & Doc** : Centralisation des éléments de conception (maquettes Figma, chartes et modélisation des données).
- **À faire / En cours / En revue** : Pilotage des fonctionnalités découpées en blocs logiques (socle technique, catalogue, tunnel de commande, espaces utilisateurs et administratifs).
- **Terminé** : Validation et clôture des fonctionnalités prêtes pour la phase de test.

3. Stratégie de Versioning et Gestion Git

La refactorisation complète du projet à la suite du changement de sujet a nécessité une adaptation de la stratégie de versioning sur le dépôt Git.

Afin de maintenir un historique propre, structuré et lisible pour le jury, la structure initiale du projet (le socle MVC, la configuration de base et les architectures de dossiers) a été initialisée proprement via une série de commits fondateurs. Par la suite, le développement s'est appuyé sur une démarche itérative standard :

- **Utilisation de commits atomiques et explicites** (ex: *Fix définitif casse dossiers (config/controllers/database/helpers/models/views)* ; *Fix CVE-2026-81525 : mise à jour mongodb/mongodb vers 2.4.2*).
- **Séparation entre une branche de production (main) et une branche de développement (dev)**, cette dernière recevant l'ensemble des évolutions avant fusion vers main une fois validées, garantissant un code stable à chaque étape de la livraison.

4. Gestion des risques techniques en déploiement

Le passage de l'environnement de développement local (Windows/XAMPP) à l'environnement de production (Heroku/Linux) a révélé plusieurs risques techniques non détectables en local, imposant une démarche de diagnostic méthodique à chaque étape :

- **Portabilité multiplateforme** : le système de fichiers Windows, insensible à la casse, masquait un décalage entre les chemins réels (Config/, Models/, Views/) et ceux attendus par le code — invisible en local, bloquant sur Heroku (Linux, sensible à la casse). Diagnostiqué via l'inspection directe de l'index Git (git ls-files), plutôt que via git status, dont le comportement peut être trompeur dans ce contexte précis.
- **Intégrité des scripts SQL** : un doublon de contrainte d'unicité (uk_email), passé inaperçu lors d'imports locaux répétés, a été rejeté par le SGBD de production (JawsDB/AWS RDS), plus strict que l'instance MariaDB locale.
- **Dépendances d'environnement** : l'extension PHP mbstring, activée par défaut sous XAMPP, nécessitait une déclaration explicite (ext-mbstring) dans composer.json pour être disponible sur le buildpack Heroku — son absence provoquait une erreur fatale sur les fonctionnalités de recherche.
- **Veille sécurité applicative** : l'exécution systématique de composer audit avant chaque déploiement a permis d'identifier une CVE affectant la bibliothèque mongodb/mongodb, analysée puis corrigée par mise à jour de dépendance, malgré un risque d'exploitation jugé nul dans le contexte d'usage du projet.

- **Cohérence du schéma relationnel** : l'absence d'auto-incrémentation sur une clé primaire (vg_menu.menu_id), non détectée avec les données de test (identifiants insérés explicitement), s'est révélée lors de la création d'une donnée réelle en production.

Cette phase a consolidé une méthodologie de diagnostic reproductible : isolement du symptôme via les logs serveur (heroku logs --tail), analyse du code source concerné, correction ciblée, puis validation en production avant propagation vers la branche de développement.

5. Rétroplanning Réel

La nécessité de refactoriser l'ensemble du projet suite au changement de sujet a imposé une organisation séquentielle rigoureuse:

- **Phase 1: Conception, Maquettage et Modélisation**
 - Analyse du nouveau cahier des charges.
 - Réalisation des wireframes et conception de la base de données (MCD MariaDB à 15 tables et collections MongoDB).
- **Phase 2 : Développement du Socle Technique**
 - Initialisation de l'environnement local (XAMPP, PHP 8.2), mise en place d'un autoloader custom (App\Autoloader) et sécurisation du routage.
- **Phase 3 : Fonctionnalités Métiers et Tunnel de Commande**
 - Implémentation du catalogue, des filtres et du parcours de commande en 6 étapes (0 à 5) avec intégration de l'API OpenRoute Service.
 - Développement des espaces employés et administrateur (gestion des stocks, modération, statistiques).
- **Phase 4 : Sécurité, Tests et Statistiques NoSQL**
 - Connexion au cluster MongoDB Atlas pour le suivi analytique.
 - Audit de sécurité (protection contre les injections, les failles CSRF/XSS) et corrections des anomalies.
- **Phase 5 : Déploiement et Documentation**
 - Finalisation de la documentation technique et de ce dossier de gestion de projet.
 - Vérification de la procédure de déploiement en production.

6. Gestion des Risques et Solutions Apportées

L'apparition de contraintes techniques et calendaires a conduit à l'application de choix précis :

- **Le changement de sujet en cours de parcours :**
 - *Contrainte* : Compression significative du temps disponible avant l'échéance de l'ECF.
 - *Réponse* : Hiérarchisation stricte des fonctionnalités pour cibler un produit minimal viable (*MVP*) pleinement opérationnel, couplée à une réutilisation ciblée du code socle déjà validé.
- **La dépendance aux services externes (APIs et base NoSQL) :**
 - *Contrainte* : Risque de rupture de service ou de latence sur les appels tiers (géolocalisation, e-mails).
 - *Réponse* : Implémentation de mécanismes de repli (*fallback* via la formule de Haversine en cas de dépassement de délai de l'API de routage) et gestion asynchrone non bloquante pour les notifications par e-mail.